

移动缓存内计算的多维矢量ISA扩展

Alireza Khadem[†], Daichi Fujiki[‡], Hilbert Chen[†], Yufeng Gu[†], Nishil Talati[†], Scott Mahlke[†], and Reetuparna Das[†]

University of Michigan[†], Institute of Science Tokyo[‡] {arkhadem,cyaa,yufenggu,talatin,mahlke,reetudas}@umich.edu, dfujiki@artic.iir.isct.ac.jp

Abstract—In-cache计算技术将现有的缓存转化为长矢量计算单元，为移动cpu构建昂贵的矢量引擎提供了低成本的替代方案。不幸的是，现有的长向量指令集架构(ISA)扩展，如RISC-V向量扩展(RVV)和Arm可扩展向量扩展(SVE)，只提供一维跨行和随机内存访问。虽然这对于典型的矢量引擎来说已经足够了，但它无法有效地利用缓存内矢量引擎的单指令多数据(SIMD)宽度。这是因为移动数据并行内核在单个维度上暴露了有限的并行性。

在分析移动向量核的基础上，提出了一种用于移动缓存内计算的长向量多维向量ISA扩展(MVE)。MVE实现了较高的SIMD资源利用率，并通过抽象缓存几何形状和数据布局实现了灵活的编程。所提出的ISA具有多维跨行和随机内存访问以及有效的维度级掩码执行来编码多维并行性。通过使用广泛的数据并行移动工作负载，我们证明了与商用移动处理器的SIMD单元相比，MVE提供了显著的绩效和能耗降低优势，平均为2.9倍和8.8倍，面积开销为3.6%。

I. 介绍

数据并行内核在各种移动应用领域无处不在，从操作系统(例如，Android)到web应用程序(例如，Chrome)[49]。这些内核通常由紧密循环组成，交错进行标量和矢量处理。单指令多数据(SIMD)管道与CPU核心集成，以利用这些应用程序的细粒度数据级并行性。SIMD执行分摊指令获取和解码的成本，以提高数据并行应用程序的绩效和能耗，即资源有限的移动设备的两个主要关注点。

虽然移动应用程序受益于矢量架构，但扩展矢量寄存器文件和alu会给移动处理器带来显着的面积和功耗开销。因此，所有Arm移动处理器的实现都被限制为128位矢量单元。另外，各种sram内计算方案[8]-[10], [31], [46], [87]重新利用缓存阵列来构建高吞吐量和节能的矢量引擎

以最小的面积开销。例如，移动CPU中小型私有L2缓存(256 KB)的一半大小可以转换为8192车道的位串行矢量引擎，其面积开销为3.5%。缓存SRAM阵列提供计算能力，也充当矢量寄存器。此外，当不用于sram内计算时，它们可以作为常规缓存使用，没有绩效开销。

现有的长向量ISA扩展(例如，RISC-V RVV[5]和Arm SVE[80])提供一维跨行和随机存储器访问。虽然对于典型的矢量引擎(128 - 1024位)来说已经足够了，但它们无法有效地利用具有大SIMD宽度(8192×32位)的长矢量缓存内矢量引擎。使用仅具有一维内存访问的传统长向量isa需要几个向量操作指令来加载/存储和解包/打包多个维度的数据结构到单个长向量寄存器。这大大增加了指令计数和核心-缓存通信，充分利用了缓存内SIMD通道。

这个问题由于移动向量内核暴露有限的一维(1D)并行性而变得更加复杂。我们的评估显示，在移动处理器[49]的矢量基准套件中，一维并行度平均为635。该基准涵盖了来自不同领域的移动应用程序，如Chromium[2](浏览器和OS)、Android[1](OS)、PDFium [4] (PDF渲染引擎)和WebRTC[6] (用于会议平台的实时语音/文本/视频通信api，如Zoom、Microsoft Teams、Slack或谷歌Meet)。例如，WebRTC的音频处理模块(webaudio[37])同时处理多个音频块，每个音频块包含128个音频样本的不同通道，这将一维并行性限制为仅128个元素。

这种观察激发了MVE，一种用于移动缓存内计算的长向量多维向量ISA扩展。所提出的ISA提供了多维跨行和随机内存访问，以及有效的维度级掩码执行。这种灵活的指令集将暴露给缓存内向量引擎的数据级并行性(Data Level Parallelism, DLP)提高到移动内核的多个维度。更高的DLP允许使用少量矢量指令利用所有8192个缓存内SIMD计算通道。

为了实现所提出的MVE ISA，我们设计了一个具有计算能力的缓存架构，该架构针对小型私有L2缓存，以便与核心紧密集成。提出的体系结构

利用核心和L2缓存之间的细粒度通信，在标量核心和缓存内矢量引擎之间交错排列标量和矢量指令。因此，MVE为处理细粒度数据并行移动核提供了缩放向量单元的替代方法。

本文提供了以下贡献：

- 我们提出了一个长向量ISA扩展，该扩展使用多维跨越式和随机存储器访问以及有效的维度级屏蔽来编码多维并行性。与RISC-V RVV[5]相比，该指令集将缓存内向量引擎的利用率从23%提高到60%，绩效平均提高3.8倍。
- 我们设计了多维矢量ISA缓存架构。针对所提出的架构，我们分析和评估了不同的In-SRAM计算方案，包括位串行[31]、位并行[9]、位混合[10]和关联计算[19]。
- 我们在十二个实际移动库[49]上验证了我们的解决方案的有效性。采用长矢量位串行缓存内计算模型的端到端系统设计将商用矢量cpu的绩效和能量分别提高了2.9倍和8.8倍。
- 我们将MVE缓存内矢量处理与移动GPU进行比较。我们的测量表明，特定领域的加速器(gpu和dsp)受到内核启动和数据复制开销的影响。这对于细粒度数据并行性至关重要，这得益于与核心紧密集成的长向量引擎。
- 我们分析并发现，由于高计算延迟和大量寄存器溢出和填充，为服务器类缓存内计算(例如Duality Cache [35])提出的粗粒度单指令多线程(SIMT)模型对于移动内核是低效的。

II. 后台

A. 长向量处理

长向量ISA扩展。表1比较了最先进的长向量ISA扩展，即RISC-V向量扩展(RVV) [5]， Arm可扩展向量扩展(SVE) [80]和NEC[53]。SVE和NEC支持高达2K位和16K位的矢量长度。在缓存内长向量引擎中使用SVE和NEC需要大量指令来针对所有8K 32位SIMD通道，从而增加了核心-缓存通信。

RVV支持一维跨行向量访问。NEC提供二维跨行负载和存储限制为恒定16×16输入矩阵大小。为移动应用程序的小型1D DLP量身定制的高效长向量ISA扩展需要多维和灵活的跨行内存访问。

RVV提供了从基址和存储在索引向量寄存器中的预定偏移量的随机存储器访问。SVE支持随机基数访问和随机偏移量访问。这些指令在捕获移动内核中的随机内存访问时效率不高，移动内核通常在最高维度有多个唯一的基址，然后是较低维度的跨行模式。

表1向量ISA扩展比较

Feature	MVE (This Work)	RISC-V RVV [5]	Arm SVE [80]	NEC [53]
Max Vector Length	infinite	infinite	2048 bits	16384 bits
Strided Access	Flexible 4D	Flexible 1D	-	Constant 2D
Random Access	Random Base + Strided Offset	Random Offset	Random Base / Random Offset	-
Masked Execution	Predicate / Dimension-Level	Predicate	Predicate	Predicate

最后，所有三种长向量ISA扩展都支持不规则DLP模式的预测执行。然而，为8K SIMD通道生成带有标量或长延迟缓存内向量指令的谓词掩码是昂贵的。我们为多维访问模式引入了一种粗粒度维度级屏蔽方法。

长向量引擎由工业界[22]、[27]、[53]、[74]、[77]、[78]和学术界[15]、[20]、[62]、[63]、[68]、[75]共同设计和制造。这些单元多为面积和功率资源丰富的超级计算机所采用。例如，Vitruvius+[63]是一种基于rvv的协处理器，具有16K位矢量寄存器。矢量单元有8个双精度或16个单精度SIMD通道。每条指令以向量方式在64个周期的SIMD通道上调度。该协处理器在22nm工艺下的面积为1.3mm² (7nm工艺下的面积为0.4mm²)，这与已评估的Arm Cortex-A76内核(7nm工艺下的面积为1.2mm²)相比是显著的，包括两个128位Neon ASIMD单元和专用L1和L2缓存。

由于长矢量引擎的高面积开销，自引入Arm Neon技术以来，所有Arm应用处理器都采用了128位ASIMD单元。MVE利用未开发的私有L2缓存资源，并将它们重新用于每个核心专用的8K×32缓存位串行SIMD通道，内核的面积开销为3.6%。

B. 缓存内计算

缓存内计算[8]，[31]，[34]，[47]，[48]将缓存SRAM阵列重新定位为SIMD单元。图1(a)显示了一个移动核心，其中一半的L2缓存被重新用于缓存内计算。我们构建的神经缓存[31]建议数据元素垂直对齐SRAM位线。SRAM外设对所有数据元素并行计算位串行操作，如图1(b)所示。因此，具有256位线的SRAM阵列提供256个SIMD通道，而具有32个SRAM阵列的256KB L2缓存片可以包含8192车道的位串行矢量引擎。在本节中，我们描述了先前的SRAM内计算方案的体系结构：

(a)位串行计算(Bit-Serial Computing, BS)[31]在位线上垂直对齐数组元素。通过激活SRAM阵列的两个字行，外围逻辑的感测放大器(SA)计算两个输入阵列中所有元素的位片的逻辑与与非。使用额外的逻辑(图1(c)的蓝色组件)来计算位片的NAND, OR和XOR。执行一个n位整数加法

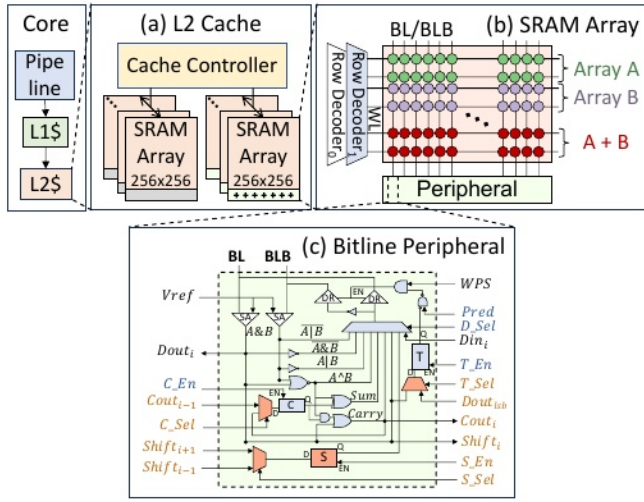


图1. (a)一半L2缓存启用了缓存内计算的移动核心。(b) SRAM内计算使用一个额外的行解码器激活SRAM阵列的两个字行。(c)位线外设的位串行(蓝色)、位混合和Bit-Parallel(蓝色+橙色)修改。

在 n 个周期内使用额外的逻辑来计算和并行执行。进位锁存器(C)将进位存储在位线外设中, 作为下一位的进位。通过使用设置为1的进位锁存(C)减去 B 并将 A 和 B 相加, 在 $2n$ 个周期内计算一个2的补减法。整数乘法则需要 $n^2 + 5n$ 个循环。首先, 乘法器的每个 i th位存储在标签锁存器(T)中, 作为位线驱动因素的启用信号。接下来, 乘法器有条件地加入到结果中, 从第 i 位开始。通过读取输入位片并写入移位结果的位片, 用定义的移位值偏移, 在 n 个周期内支持恒定移位操作。可变移位需要 $O(n \log n)$ 个周期[35]才能将 B 的每个 i th位读入 T 锁存器, 并使用 2^i 位值执行条件常数移位。对偶缓存[35]将这些整数运算扩展为浮点算术运算。

(b) Bit-Parallel [9] (BP)和位混合 [10] (BH)。BS将并行性最大化到8K SIMD通道, 但具有很高的算术延迟。VRAM[9]引入了BP, 在字行中水平对齐 n 位数据。这将并行性降低到 $8K/n$ 的SIMD通道, 但将延迟提高了 n 倍。EVE [10]通过合并BS和BP方案引入BH, 以平衡延迟和并行性。BH将 n 位数据分割成 p 位数据段, 这些数据段使用BP计算, 并使用BS进行组合。BP和BH计算需要跨位线外设进行通信, 这会产生面积和频率开销。图1(c)用橙色显示了位线间通信的附加逻辑。EVE[10]采用曼彻斯特携带链, 路由执行到下一个位线, 用于段内的完整添加, 而C触发器用于段之间的添加。

(c) AC计算(Associative Computing)[19]。与先前的方案相比, AC不需要在位行外设中使用额外的逻辑来进行逻辑操作。相反, AC使用二进制内容可寻址存储器(BCAM)结构[46]的搜索和更新操作。为了执行任何逻辑操作, AC对真值表的每一行执行搜

索和更新。使用AC, CAPE[19]水平对齐字行中所有数组元素的位片。由于CAPE在不同的SRAM阵列中分配不同的位片, 因此它在 $O(1)$ 个周期内对所有位并行执行逐位操作。先前的工作[18]利用了这些按位逻辑操作的低延迟来加速数据库工作负载。然而, 由于它需要在位片之间进行进位传播, 因此搜索和更新操作是对完全加法的不同位顺序执行的。一个整数减法/加法需要 $8n+2$ 个循环。由于其他算术运算被分解为加法和减法, 它们也会产生很高的操作延迟。

总之, 所有In-SRAM计算方案都将缓存SRAM阵列转换为高吞吐量长向量引擎。因此, 需要MVE利用移动内核的多维DLP来处理大量的缓存内SIMD通道。虽然第VII-B节评估了这一说法, 但我们使用位串行计算来解释我们在本文中的贡献。

III. 指令集架构

我们借鉴了分析Swan[49]的见解, Swan[49]是用于移动处理器的最先进的数据并行应用程序基准套件。我们发现多维数据结构在移动应用中很普遍。这些见解为我们的设计选择提供了信息, 使我们能够在拟议的MVE扩展中有效地利用移动工作负载特征。

A. 设计目标

考虑到缓存内计算引擎的高吞吐量, ISA设计的关键挑战是通过开发一个紧凑的ISA来实现高的计算资源利用率, 该ISA可以在很少的指令中表达向量计算。生成大量动态指令的ISA会导致指令问题瓶颈和微体系结构模块溢出。因此, 标量核不能发出足够的指令来保持缓存中的矢量单元忙碌。

此外, 先前的缓存内ISA扩展[8], [54], [79]向程序员公开了缓存的几何形状和数据布局, 要求他们沿着特定的SRAM位线加载和对齐数据元素, 以确保操作数的局部性。这种方法增加了编程的复杂性, 因为程序员必须处理缓存库、集合、方式、字行和位线坐标, 导致二进制文件与其他缓存体系结构不兼容。相比之下, 我们的目标是通过将缓存内计算呈现为长向量处理引擎来抽象这些复杂性。程序员可以使用与其他长向量扩展类似的缓存内向量指令和寄存器, 而底层架构则透明地将长向量寄存器映射到SRAM位线和字行坐标。

B. 物理寄存器抽象(Physical Register Abstraction)

图2显示了缓存中矢量寄存器的数据布局:(a) MVE支持 N 个物理寄存器(PR_0 到 PR_{N-1})。 (b)在内部, 物理寄存器的数据元素被调换并垂直放置在位线上, 以支持位串行In-SRAM操作。因此, 具有256的SRAM阵列

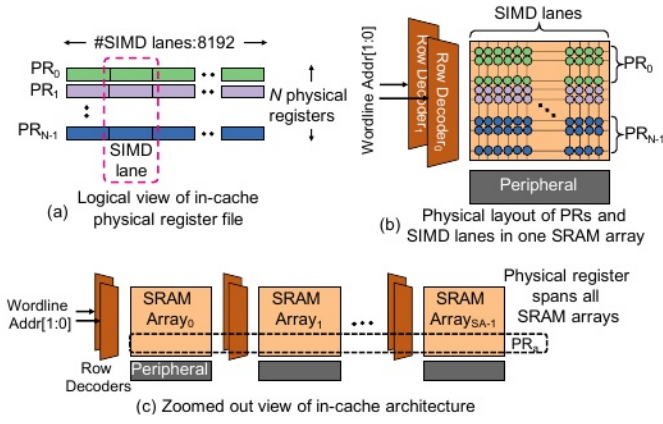


图2. (a) MVE在N个长向量缓存寄存器上操作。(b)缓存内数据元素和SIMD通道使用位线的垂直数据布局。(c)一个高速缓存内物理寄存器覆盖所有可计算的SRAM阵列。

bitlines为PR提供256个数据元素。(c)每个PR跨越所有SRAM阵列。因此，缓存中的长向量PR包含使用32个SRAM阵列的8K数据元素。

我们观察到Swan基准套件[49]的移动数据并行内核使用嵌套循环对多维数据结构进行操作，并最多利用四个维度。此外，移动数据并行核通常在较小的输入数据大小上运行，因此一维不足以利用缓存内向量引擎的吞吐量。因此，MVE ISA不是在线性地址空间(即PR_i[s])中对每个8K SIMD通道进行寻址，而是将物理寄存器视为多维逻辑寄存器(即PR_i[w][z][y][x])。

MVE缓存控制器(第V节)在特定的控制寄存器(CR)中维护矢量引擎配置。程序员使用设置cr的配置指令来选择尺寸计数和长度。逻辑寄存器索引被平展到8K SIMD通道，并由MVE控制器映射到MVE物理寄存器。许多内核对多个输入批次使用相同的内存访问模式。因此，配置cr的开销在多个矢量执行中平摊。

传统矢量体系结构中的物理矢量寄存器文件具有固定数量的、宽度恒定的寄存器，如Vitruvius+[63]体系结构中有40个16K位寄存器。向量ISA扩展，如RVV[5]，提供恒定数量的寄存器，但根据数据精度改变向量长度。相比之下，In-sram计算的位串行布局提供恒定的矢量长度(8K位行)，但允许基于数据宽度的可变数量的寄存器。尽管不考虑数据精度的固定向量长度简化了向量化，但可变数量的寄存器使寄存器分配复杂化(第III-G节)。

MVE计算指令在物理寄存器的所有SIMD通道上执行相同的操作，但是对物理寄存器的加载和存储可以针对不同的内存位置。正如我们接下来讨论的那样，多维逻辑向量寄存器抽象允许卸载通用和多维内存访问模式。

C. 具有可能复制的跨越式内存访问

数据并行核的内存访问遵循一个规则的模式:嵌套循环处理多个数据维度，每次迭代的基本内存地址以一个恒定的步幅不同。因此，我们提出了如算法1所示的多维跨步内存访问指令。

Algorithm 1 Strided Multi-dimensional Vector Load

Input: $Dim_{[3:0]}$: Length: CRs, Base Address, and $S_{[3:0]}$: Strides

```

1: for w in [0 : Dim3.Length) do
2:   for z in [0 : Dim2.Length) do
3:     for y in [0 : Dim1.Length) do
4:       for x in [0 : Dim0.Length) do
5:         PRi[w][z][y][x] = MEM[Base + w × S3 + z × S2 + y × S1 + x × S0]

```

例如，图像和视频编码的图像内预测核[83]中使用的典型模式如图3所示。该内核需要S₀= 1, S₁= 0, S₂= 3的3D跨行矢量加载。输入数据位于2D数据结构中，但在实际计算时被加载到3D逻辑寄存器中。跨行矢量加载指令对从2D加载到3D的数据进行编码，并在单个指令中进行复制。跨步S₁= 0便于将输入数据行中的每个元素复制到3D逻辑寄存器中的整个列。该图进一步显示了从3D逻辑寄存器表示到物理寄存器的不同SIMD通道的平坦数据映射。

一维矢量ISA扩展，如RVV[5]，将使用6个跨行加载指令，分别访问输入数据的每一行(例如，[0][1][2])到向量寄存器的3元素段，同时屏蔽其他段。需要进一步的标量指令来计算掩码，并需要6条指令来加载它们。MVE在单个指令中编码此模式，而不需要任何掩码。

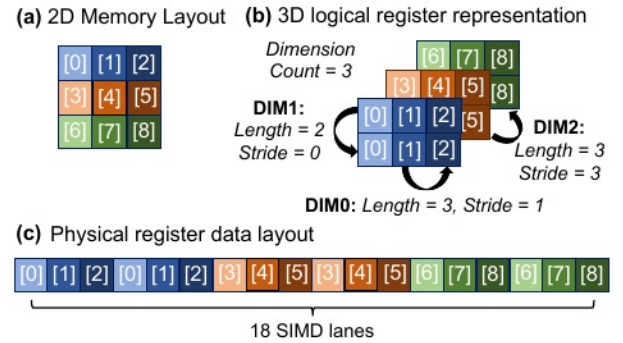


图3. 图像内预测内核的跨行存储器访问示例:从(a) 2D存储器布局加载到(b) 3D逻辑寄存器，通过MVE控制器映射到(c)平坦物理寄存器的SIMD通道。

每个跨步值(S_i)最多占用16个指令位。对不同维度的多个步长值进行编码会增加指令宽度。然而，我们的分析表明，步长值为0(用于复制)和1(用于顺序访问)是经常使用的。因此，我们不再使用16位的绝对跨距值，而是为每个维度编码一个2位的跨距模式(四维为8位)。模式0和模式1编码的是常用模式

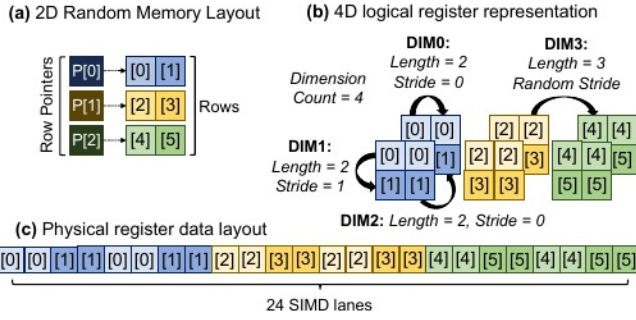


图4。h2v2上行样本内核的随机内存访问:从(a)随机行指针加载到(b) 4D逻辑寄存器。(c)显示了平坦的物理寄存器的SIMD通道。

步幅值为0和1。模式2是为顺序访问设计的，其中高维的步幅不同。例如，从逐行2D矩阵加载时，行维的步幅值等于列数。因此，对第*i*维使用模式2可以得到 $S_i = S_{i-1} \times \text{Dim}_{i-1} \cdot \text{length}$ 。如果期望的跨步值不是上述两种模式，我们为MVE控制器中的每个维度提供一个加载和存储跨步CR，这是由配置指令设置的。模式3使用这些CR值。

D. 带跨行和复制的随机内存访问

我们观察到，在一些移动数据并行内核中，最外层的循环(最高维度)需要唯一的基址，而内部循环则遵循跨步模式。图4用libjpeg库[57]的例子演示了这一概念。Libjpeg在不同的内存位置分配行。h2v2上行样本内核将每个像素水平和垂直复制为4个像素。虽然第4维中的地址是随机的(行指针)，但其他维度遵循步长值0(垂直复制)，1(行像素)和0(水平复制)。

MVE的随机存储器访问指令对输入指针数组的地址进行编码。MVE控制器从该数组中获取随机基址，并对内部维度使用步长值，如算法1所示。

$$PR_i[w][z][y][x] = MEM[Base_w + z \times S_3 + y \times S_2 + x \times S_1] \quad (1)$$

RVV[5]支持单个基址，其随机偏移量位于矢量寄存器中。用这些指令实现上述内存访问模式需要每个基址3个掩码随机内存访问，计算掩码的标量指令，以及6个额外的内存访问来加载掩码和偏移向量寄存器。相比之下，MVE在没有标量指令开销的单个随机内存访问中对该模式进行编码。

E. 高效掩码执行

MVE使用两种类型的指令支持屏蔽执行:(a)常规预测执行:MVE提供比较操作，其结果存储在标签锁存器(T)中，以独立地屏蔽每个SIMD位线(图5中维度树的叶子)。(b)维度级屏蔽操作:我们观察到，大多数数据并行核

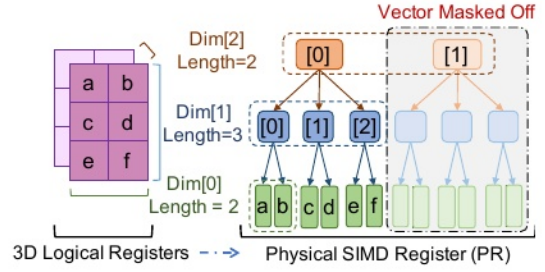


图5。MVE控制器映射多维逻辑寄存器到一维物理SIMD寄存器。高效的维度级掩码执行掩码掉树最高维度节点下的叶子(最外层循环的迭代)。

需要粗粒度屏蔽执行，其中与最外层循环对应的SIMD通道被屏蔽掉(橙色节点[1]下的叶子)。我们保留一个掩码CR，它包含最高维的每个元素一个比特。我们将最高维度的最大长度设置为256，以限制掩码CR大小。掩码值由标量指令计算，并在MVE控制器中使用矢量掩码指令设置，该矢量掩码指令掩码打开/关闭最高维的特定元素的SIMD通道。使用传统的谓词执行来实现这种语义需要计算标量核中每个元素的掩码值，将其存储到内存中，并将其加载到矢量掩码寄存器中。维度级掩码操作是高效的，因为它们不使用In-SRAM物理寄存器或标量指令。

F. 编程模型

MVE支持8/16/32/64位un/signed整型和16/32位浮点型数据类型，后缀为b/w/dw/qw和hf/h。每种数据类型实现了29个运算，分类见表二。配置操作设置的是MVE控制器的cr，不能在SRAM阵列上执行。Move操作用于转换和复制寄存器。Load和Store操作提供跨行和随机的向量内存访问。算术运算用于根据源寄存器和直接值设置寄存器的值。

表二MVE指令

Type	Operation	Assembly	BS Latency*
Config	set dim count	<i>vsetdimc rs</i>	—
	set dim length	<i>vsetdiml rs1 rs2</i>	
	(un)set mask	<i>v(un)setmask rs</i>	
	set kernel width	<i>vsetwidth imm8</i>	
Move	convert	<i>vcvt vd vs</i>	<i>n</i>
	copy	<i>vepy vd vs</i>	
Memory Access	strided load	<i>vslld vd rs1 rs2</i>	—
	random load	<i>vrlld vd rs1 rs2</i>	
	strided store	<i>vsst vs rs1 rs2</i>	
	random store	<i>vrst vs rs1 rs2</i>	
Arithmetic	set duplicate	<i>vsetdup vd rs</i>	<i>n</i>
	shift immediate**	<i>vshi vd vs rs</i>	<i>n</i>
	rotate immediate**	<i>vroti vd vs rs</i>	<i>n</i>
	shift register**	<i>vshr vd vs1 vs2</i>	<i>nlog(n)</i>
	addition	<i>vadd vd vs1 vs2</i>	<i>n</i>
	subtraction	<i>vsub vd vs1 vs2</i>	<i>2n</i>
	multiplication	<i>vmul vd vs1 vs2</i>	<i>n² + 5n</i>
	min/max	<i>vmin/max vd vs1 vs2</i>	<i>2n</i>
	xor	<i>vxor vd vs1 vs2</i>	<i>n</i>
	comparison	<i>vgt(e)/lt(e)/(n)eq vs1 vs2</i>	<i>n</i>

*计算基于精度(n)的位串行有符号整数操作的延迟。 **移位和旋转操作包括右(r)或左(l)变体。

为了在C/C++程序中使用MVE数据类型和指令，我们开发了一个支持变量和内在函数的库。MVE变量被声明为`__mdv`，作为多维变量的首字母缩略词，由数据类型后缀连接。程序员将这个库包含在代码中，并且使用MVE开发他们的内核，与其他长向量架构(例如，RVV[5]和SVE[80])所需的级工作量相当。我们开发了一个内部编译器基础结构，并在数据并行MVE内核上应用了以下编译器^[8-9]优化。可^{执行二进制文件}在核心的前端模块中解码，¹¹个标量后端管道执行标量指令，长⁻¹²

向MVE控制器发出矢量缓存指令

G. 编译器

编译器对MVE的支持类似于长向量架构，但需要考虑以下因素：

寄存器计数。虽然传统架构定义了固定数量的pr，但MVE pr受到字行数(256)和活动pr宽度的限制(图2(b))。为了简化寄存器分配，我们为内核采用单一数据宽度。使用liveness Analysis [32]，编译器检测内核最宽的PR宽度，并注入配置指令(表2)将PR宽度设置为控制寄存器。MVE控制器使用此CR将pr映射到字行。

寄存器分配和指令调度是两个代码生成优化，可以改善寄存器溢出和ILP。我们将重点放在矢量寄存器压力上，因为由于pr数量有限以及宽缓存内矢量寄存器的高溢出成本，它对MVE至关重要。我们使用列表混合指令调度程序[60]，这是一种自下而上的列表调度算法，它试图使活动寄存器小于目标机器的PR计数，并通过调度不产生寄存器溢出的指令来缩短寄存器的活动范围。此外，我们采用贪婪寄存器分配[67]，通过广泛分割活动范围来最大限度地减少寄存器溢出，减少寄存器干扰。

IV. 常见的数据并行模式

为了说明MVE ISA的可表达性，我们展示了¹¹个评估的移动内核的通用数据并行模式的实现。这些指令列在表II¹³中

与数据类型后缀连接。

矩阵转置在机器学习(XNNPACK[40])、图像/视频编解码器(Kvazaar[86]和libjpeg[57])和密码学(boringssl[38])库中使用。下面的代码片段显示了使用MVE对 $M \times N$ 矩阵的变换。为简单起见，在本例中，我们假设两个维度都小于SIMD通道的数量。任何内核都是从配置维度数、长度(第3行)和步幅cr (L5)开始的。转置内核的主循环使用步幅值N和1 (L9)垂直加载输入矩阵列，并以步幅值1和N (L12)水平存储输出矩阵行。为了转置 512×49 矩阵(MobileNet-V1[43]内核之一)，M

VE实现只需要4次迭代，而1D实现需要49次迭代来分别加载/存储输入列。

```
void transpose(int input[M][N], int output[N][M]) {
    // 2D: M output columns (DIM0), 8192/M rows (DIM1)
    vsetdimc(2); vsetdiml(0, M); vsetdiml(0, 8192/M);
    // load stride (DIM0) = N, store stride (DIM1) = N
    vsetldstr(0, N); vsetststr(1, N);
    for (int i = 0; i < N; i += 8192/M) {
        // LD 8192/M columns: stride (DIM0) = MODE3 (N)
        //                               stride (DIM1) = MODE1 (1)
        __mdvdw in_val = vlds_dw(&input[i], 3, 1);
        // ST 8192/M rows: stride (DIM0) = MODE1 (1)
        //                               stride (DIM1) = MODE3 (N)
        vsts_dw(&output[i*M], in_val, 1, 3);
    }
}
```

约简在Network (Checksum[12])、音频/图像处理(libwebp[59]和webaudio[37])和压缩(zlib[7])库中使用。将输入数组减少到8K元素(SIMD通道数)只需使用1D矢量加载和添加即可实现。超过8K元素的减少重复将向量寄存器分成两半，然后减少到一半的元素。我们不支持SRAM阵列和位线之间的任意数据移动来对齐半寄存器，而是使用维度级掩码操作(L6)屏蔽掉寄存器的前半部分，并使用2D向量存储(L8)将向量寄存器的后半部分存储到内存中。接下来，我们将向量长度设置为一半(L10)，将另一半加载到临时寄存器(L11)中，并减少一半向量寄存器(L13)。我们重复这个垂直缩减步骤，直到我们将数组减少到256个元素(5步)，之后在CPU核心中进行缩减，因为由于显着的算术运算延迟，在此之后缓存内计算是无益的。

```
__mdvdw vertical_reduction_step(__mdvdw a, int M) {
    int tmp_mem[M];
    // Split M SIMD lanes into 2 M/2-element halves
    vsetdimc(2); vsetdiml(1, 2); vsetdiml(0, M/2);
    // Mask off the first half (element 0)
    vunsetmask(0);
    // Store the second half to the temp memory
    vsts_dw(tmp_mem, a, 1);
    // Load the second half into a register
    vsetdimc(1); vsetdiml(0, M/2);
    __mdvdw tmp_val = vsld_dw(&tmp_mem[M/2], 1);
    // Reduce the two M/2-element registers
    return vadd_dw(a, tmp_val);
}
```

不规则访问需要随机向量加载和存储。我们在两种情况下观察到这种模式:(a)稀疏计算:例如，在XNNPACK[40]库的SPMM内核中，输入矩阵是稀疏的，并以CSR格式压缩。Core根据行指针计算非零输入单元地址，MVE随机加载并水平复制它们。类似地，core计算非零输入单元对应的权重行地址，MVE按顺序随机加载它们。(b)数据指针:图像处理库(libjpeg[57]、libpng[58]和libwebp[59])和图形库(Skia[39])将帧行维护在随机的内存位置。对多行进行操作

同时，我们需要从随机基址加载或存储不同的行。下面的代码片段只展示了libjpeg的upsample内核的内存访问模式。输入像素从随机输入行指针加载，每个像素被复制两次(L10)。输出结果依次存储到输出指针中(L14)。

```

1 void upsample(char* input[M], char* output[2M]) {
2     // 3D: replicate 2 pixels (DIM0)
3     vsetdimc(3); vsetdiml(0, 2);
4     // M columns (DIM1), 8192/(2M) rows (DIM2)
5     vsetdiml(1, M); vsetdiml(2, 8192/(2M));
6     for (int n = 0; n < N; n += 8192/(2M)) {
7         // Random LD rows from input pointers
8         // Load pixels in a row: stride(DIM1)= MODE1(1)
9         // Rep. pixels twice: stride(DIM0)= MODE0(0)
10        __mdvf rows_val = vrdl_b(&input[n], 0, 1);
11        // Random ST rows to output pointers
12        // Store sequentially: stride(DIM0) = MODE1(1)
13        //                               stride(DIM1) = MODE2(2)
14        vrst_f(&output[n], rows_val, 1, 2);
15    }
16 }

```

多维复制在所有数据并行内核中都很流行。例如，XNNPACK[40]库的逐行GEMM内核分别水平(DIM0)和垂直(DIM1)加载和复制输入(L12)和权重(L15)元素。将 i^{th} 步幅设置为零，跨 i^{th} 维度进行复制，如下面的代码所示。

```

1 void GEMM_Replication(float input[N][K],
2     float weight[K][M], float output[N, M]) {
3     // 2D: M output columns(DIM0), 8192/M rows(DIM1)
4     vsetdimc(2); vsetdiml(0, M); vsetdiml(1, 8192/M);
5     // load stride(DIM1)= K, store stride(DIM1)= M
6     vsetldstr(1, K); vsetststr(1, M);
7     for (int n = 0; n < N; n += 8192/M) {
8         __mdvf acc_val = vsetdup_f(0);
9         for (int k = 0; k < K; k++) {
10            // LD an input column: stride(DIM1)= MODE3(K)
11            // Rep. horizontally: stride(DIM0)= MODE0(0)
12            __mdvf input_val = vsld_f(&input[n*K+k], 0, 3);
13            // LD a weight row: stride(DIM0)= MODE1(1)
14            // Rep. vertically: stride(DIM1)= MODE0(0)
15            __mdvf weight_val = vsld_f(&weight[k*M], 1, 0);
16            acc_v = vadd_f(acc_v, vmul_f(input_val, weight_val));
17        }
18        // ST sequentially
19        vsst_f(&output[n*M], acc_val, 1, 3);
20    }
21 }

```

V. 设计

A. 核心微架构

MVE指令被获取、解码并推送到重新排序缓冲区(ROB)和加载存储队列(LSQ)。我们不支持MVE指令的无序和投机问题。因此，LSQ防止了标量负载和MVE存储的重新排序。一旦提交，MVE指令被发布到ROB头部的L2缓存中，并从LSQ和ROB中退出。因为没有MVE指令写入标量寄存器(表2)，所以这些指令不需要回写。

然而，MVE存储被推送到写缓冲区，直到完成并被MVE控制器确认。缓存内MVE存储执行需要许多周期，这会使ROB中较年轻的标量负载停滞。为了减少不必要的延迟，我们在LSQ中提供了一个地址解码器。地址解码器维护DimCount， $Dim_{[0:3]}.Length$ 和 $Dim_{[0:3]}.Stride$ ，这些cr在向缓存发出配置指令时更新。当一个MVE存储被提交并推送到写缓冲区时，地址解码器根据公式2计算一个地址范围。使用这个地址范围，写缓冲区查找ROB中的标量负载和写缓冲区中的MVE存储之间的内存依赖关系。

$$Range = Base + \sum_{i=0}^3 Dim_i.Length \times Dim_i.Stride \quad (2)$$

B. 缓存架构。

图6以粉红色显示了对缓存的新修改，以启用MVE的执行：

MVE控制器按程序顺序从核心接收MVE指令，并将其排队到Instruction-Q中。在解码配置指令时，MVE控制器设置的CR值与核心中的地址解码器相似。逻辑寄存器根据维度长度被平展到物理寄存器，映射到数据数组，并发布到有限状态机(FSM)。FSMs进一步将指令解码为控制SRAM阵列的解码器和逻辑外设的微操作(μops)。

控制块。用独立的FSM增加每个SRAM阵列会导致高面积开销。因此，我们为控制块(CB)使用四个SRAM阵列的粗粒度，并为每个CB[35]使用单个FSM。由于维度级别的掩码操作，一些CBs可以被MVE指令屏蔽掉。因此，MVE控制器通过保持一个掩码位向量来编排CBs的非阻塞执行，该掩码位向量显示每个指令必须向哪个CBs发出。如果CB未被屏蔽，则MVE控制器向该CB发出指令。

MVE控制器为每个CB保留一个PC，该PC指向Instruction-Q中的指令。当CB完成一条指令时，它通过发出ACK信号来中断MVE控制器。MVE控制器解除已完成指令的相应位向量。在增加PC时，MVE控制器跳过那些位向量为CB屏蔽的指令。当一条指令的掩码位向量全部未设置时，它将从Intrinsic-Q中退出队列。MVE ISA不支持任何寄存器到寄存器的排列或变换，SRAM阵列和位线之间的任何数据传输都是通过内存进行的(表II)。为此，我们确保在所有CBs中只有一个并行执行的加载或存储指令。因此，MVE控制器阻塞矢量存储器访问，直到所有CBs完成执行。

跨行和随机内存访问执行。在向CB发出内存访问时，MVE控制器使用基址(es)、步幅计算各个地址

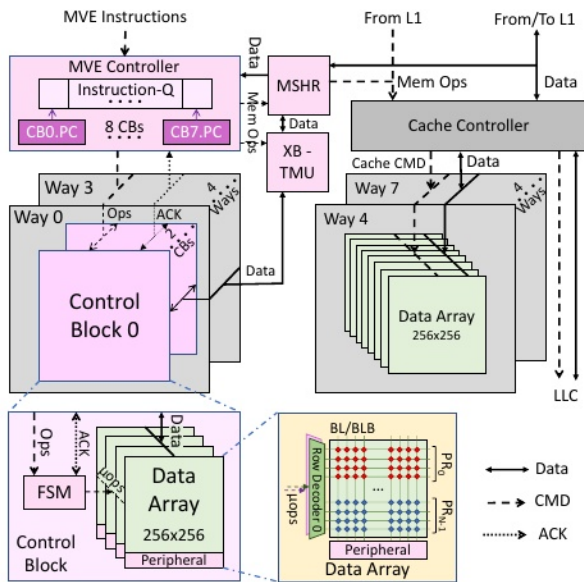


图6。MVE在缓存架构中增加了粉色模块。

模式和cr(算法1和方程1)。虽然有机会基于跨步语义合并MVE控制器中的内存访问,但我们决定简化控制器并为此目的使用Miss Status Holding寄存器(MSHR)。计算出的每个SIMD通道的地址被发送到MSHR,从L2缓存的常规一半加载,并传递到转置内存单元(TMU)。TMU由8T转置位单元组成,可以在垂直和水平方向上访问数据。TMU的大小是为了维护CB(1024个元素)的物理寄存器。使用横条(XB)将数据字路由到TMU SRAM单元中的正确垂直位置。当接收到CB的所有元素时,MVE控制器向相应的CB发出信号,从TMU水平读取寄存器位片并写入数据数组。而存储指令则要求与上述过程相反。

C. 讨论

缓存一致性。传统的缓存一致性目录为每个核心的L1和L2缓存保持单一的缓存行状态。然而，在MVE中，L2缓存为缓存内负载访问内存，并且独立于核心和L1缓存进行存储。因此，缓存级别不再是一致的。我们观察到基线L2缓存是严格包含的，并且标签数组包含每个缓存线的存在位，表明它在L1缓存[11]中是否有效。为了保持与L1缓存的一致性，我们启用MVE控制器来检查标签阵列的存在位。当缓存内内存访问的缓存命中时，MVE控制器检查存在位，如果L1缓存中设置了该缓存行，则将该缓存行从L1缓存中逐出。包容性L2缓存缺失保证数据不在L1中，从而确保L1和L2之间的一致性。

如果不需要缓存内计算怎么办?我们小心地为一半的缓存方式[35]配备了缓存内计算,同时保留了标量或SIMD应用程序的存储能力。MVE不会永久禁用一半的缓存银行以支持该计算模式。缓存库在不

用于sram内计算时作为常规缓存运行。对于不能从长向量执行中获益的工作负载，我们默认使用正常的CPU执行和常规的缓存使用。对于长向量应用程序，要从普通缓存切换到缓存内计算，MVE会刷新脏缓存线。基于50% L2缓存线为脏[50]的先验启发式，此过程所需的执行时间不到所评估基准的2%。另一方面，将计算缓存切换到普通缓存只需要修改CR，对工作负载的影响可以忽略不计。

VI. 评估方法

基准。我们使用来自Swan基准套件[49]的12个真实移动库的44个数据并行内核来评估MVE，如表III所示。为了与之前的工作进行详细的分析和比较，我们选择了11个具有各种维度和数据并行模式的内核(第IV节)。

表iii评估的库

Domain	Library	#Kernels	Dataset	Dim
Linear Algebra	Linpack [30]	1	512K	1D
Machine Learning	XNNPACK [40]	2	156 CNN Layers	2D
Signal Processing	CMSIS-DSP [14]	3	192K	1D
Video Processing	Kvazaar [83]	4	1280×720	3D
Image Processing	libjpeg [57]	5	1280×720	2-4D
	libpng [58]	3		
	libwebp [59]	7		
Graphics	Skia [39]	4	1280×720	1-3D
Audio Processing	Webaudio [37]	5	32S×44.1kHz	1-3D
Data Compression	zlib [7]	2	128KB	1-2D
Cryptography	boringsssl [38]	3	128KB	1-2D
String/Network Utilities	Arm Optimized Routines [12]	5	128KB	1-2D

移动平台。表IV显示了使用高通骁龙855 SoC的基线:封装simd (Arm Neon) 执行 Arm Cortex-A76 核心 [11] 和高通 Adreno 640 GPU。我们使用四种二级缓存方式进行数据存储, 四种缓存方式(32 SRAM阵列)用于缓存内计算。

表iv

高通骁龙855基线

Configuration	Detail
Scalar core	2.8GHz, 4-way out-of-order, 128 entry ROB
Vector engine	2 128-bit Advance SIMD units + crypto and FP16 ext
L1-I cache	64KB, 4-way, 4 cycle latency, 20 MSHRs
L1-D cache	64KB, 4-way, 4 cycle latency, 20 MSHRs
L2 cache	512KB, 8-way, Private, Inclusive, 12 cycle latency, 46 MSHRs
LLC	2MB, 8-way, Shared, Inclusive, 31 cycle latency, 64 MSHRs/way
MVE	32 8-KB SRAM Arrays, 4-SA CB, 2KB Intrinsic-Q
GPU	2 cores, 384 ALUs, 685MHz, 1MB on-chip memory

绩效建模。我们开发了一个合成的MVE内部库，模拟了6种数据类型的29个内部函数，并通过将输出与Arm Neon内核进行比较来验证功能的正确性。为了生成指令跟踪，我们实现了一个DynamoRIO客户端[42]，它将动态指令转储到Armv8.2-A服务器CPU(与移动处理器相同)上，同时用相应的操作码替换对内部库的函数调用。我们的

编译基础结构执行寄存器分配和指令调度到缓存内寄存器和MVE指令。然后，我们构建了一个跟踪驱动的、周期精确的模拟器，用表IV和第V节和图6中解释的缓存架构组件的配置忠实地模拟了Arm处理器的微架构。我们使用二元缓存[35]的位串行sram内指令延迟。我们的模拟器将内存访问注入到模拟器[51]中，以模拟内存延迟和带宽。

功率和面积建模。我们使用[31]中的位串行sram内计算能量参数，并使用CACTI评估缓存访问能量[64]。动态CPU和GPU能耗使用Batterystats[28]和Trepn profiler进行测量[70]。我们从之前的工作[31]、[35]中获取TMU、XB、FSM和Peripheral区域，并根据MVE架构需求对其进行缩放。我们在RTL中实现了控制器和地址解码器，并用15nm CMOS库进行了合成。MSHR面积采用CACTI评估[64]。标量CPU核心和GPU面积使用模射[3]，[41]估计，Arm Neon面积使用矢量引擎设计[21]进行慷慨评估。我们使用公式[81]仔细地将所有面积和能量数缩放到7nm。

先前的工作。对于二元缓存[35]，我们在CUDA中实现内核，并在GPU Ocelot模拟器[29]上运行它们以捕获动态PTX跟踪。我们使用我们的缓存内操作延迟来计算计算周期，并通过模拟器[51]模拟内存访问。对于BH、BP和AC方案，我们使用针对其SIMD通道和缓存内寄存器数量进行优化的各种算法实现内核。我们忽略了先前工作的不同工作频率和架构贡献，以证明在类似的硬件配置下，MVE对各种sram内计算方案的好处。编译器和跟踪驱动的模拟器配置了缓存内寄存器计数，每个SRAM阵列的SIMD通道数，以及基于先前工作[9]，[10]，[19]的计算和数据移动延迟。为了比较MVE与RISC-V RVV[5]，我们使用仅针对1D矢量指令的优化算法实现工作负载。

VII. results

A. 绩效与能量分析

MVE vs. Arm Neon。图7(a)显示了归一化到Arm Neon的MVE执行时间。由于篇幅有限，我们只显示一个库的所有内核的平均值。在所有被评估的移动内核中，MVE的性能比Neon高出2.9倍，这表明MVE有效地利用了缓存内计算的更高计算吞吐量。平均而言，执行时间为40.4%的空闲时间，24.7%的计算时间和34.8%的数据访问时间。MVE绩效改进在精度较低的内核中增加(参见第VII-E节)。因此，MVE在libpng、libjpeg、libwebp(图像编解码器)、Skia(图形)和Arm优化例程(字符串/网络实用程序)库上的性能明显优于Neon，这些库对8位像素值和字符进行操作。

空闲时间是指控制块(CBs)没有MVE指令要执行的时间的部分增加

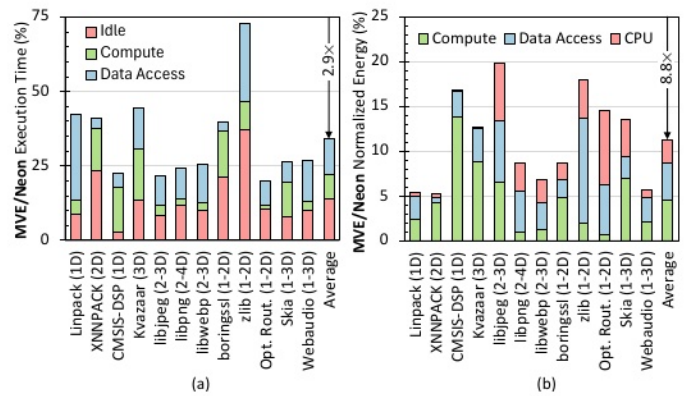


图7. (a) MVE归一化到填充simd Neon模型的执行时间和(b)能耗。

标量指令更高。例如，zlib库的Adler32内核(压缩)需要约简内核(在第四节中解释)。在基于树算法的垂直约简中，许多CBs是空闲的。标量核将元素从256减少到1，而CBs是空闲的。空闲时间使zlib的绩效提高仅为37%。

MVE平均提高8.8倍的能耗，如图7(b)所示。MVE中显著的能量降低有两个原因:(a)长向量执行模型有效地将大量操作打包到一个多维指令中，减少了CPU指令和能量。(b) sram内计算操作节能[8]。此外，缓存内计算防止数据从L2缓存传输到核心的寄存器文件并返回。

MVE vs GPU和DSP。移动应用处理器集成了GPU、DSP (NPU)、ISP和SPU等专用加速器，可加速多媒体处理、机器学习、图像信号处理和安全等特定应用领域。相比之下，MVE是一种通用的方法，可以灵活地加速所有应用领域。

图8比较了MVE与Adreno 640 GPU的执行时间和能耗。尽管32位整数MAC吞吐量降低了13.6倍，但由于两个主要原因，MVE的性能比GPU高9.3倍，能耗降低5.2倍：

(a)数据传输会给gpu带来很大的开销。为了降低这一成本，我们遵循骁龙OpenCL优化指南[71]使用统一的物理内存。虽然我们没有考虑内存分配的成本，但将数据从复杂的c++对象传输到统一内存区域中的固定C数组指针会产生大量开销，比MVE的平均执行时间长6.9倍。

(b)在扣除数据传输时间后，MVE仍然提供2.4倍的平均提速。这归因于OpenCL运行库[49]的高内核启动开销和通过系统结构的核心-gpu通信。同样，DSP也会经历内核启动开销，包括由ADSPRPC、skel和内核对象传输、存根/skel参数反/序列化以及通过系统结构[17]的FastRPC远程调用引入的延迟，[69]。相反，MVE通过利用SIMD执行模型的细粒度标量向量指令交错来避免这些开销。



图8。归一化到MVE的Adreno 640 GPU的绩效(主y轴)和能量(次y轴)消耗。

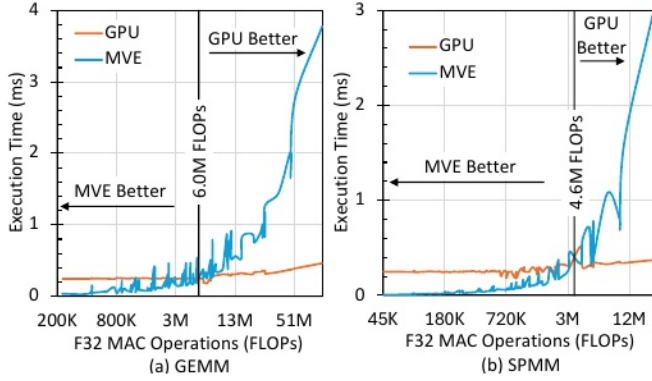


图9.MVE和Adreno 640 GPU在不同运算次数(矩阵大小)下的GEMM和SpMM执行时间。

最后，请注意，我们将单核MVE绩效与GPU进行比较。通过缓存内计算增强骁龙855 SoC的所有8核将进一步提高MVE与单个GPU相比的绩效优势。

为了更深入地比较MVE和GPU，我们从CLBlast[65]和cISPARSE[24]库中获取OpenCL GEMM和SpMM实现，并使用图9对13个移动卷积神经网络[40]的152层中使用的不同矩阵大小进行评估。MVE以6.0M和4.6M FLOPs的性能优于GPU的矩阵乘法。在大型问题中，GPU的高吞吐量抵消了其内核启动开销。另一方面，MVE只需要8K DLP就可以充分利用SIMD通道，因此它更适合较小的问题规模。

B. 多维ISA福利

图10和图11比较了MVE与一维长向量ISA扩展(如RISC-V RVV[5])在使用相同的底层位串行缓存内计算引擎时的绩效和动态指令数。我们选择了从1D到3D不同维度的各种内核进行比较。MVE显著减少了需要多维访问的内核的空闲时间，并在所有选择的内核中平均提高了2.0倍的绩效。为了使用RVV模拟多维访问，我们使用谓词操作来屏蔽一维跨行内存访问，并(un)将数据结构的每个一维段分别打包在一个向量寄存器中。这种方法为每个多维内存访问生成长度(1D段)配置(掩码)，部

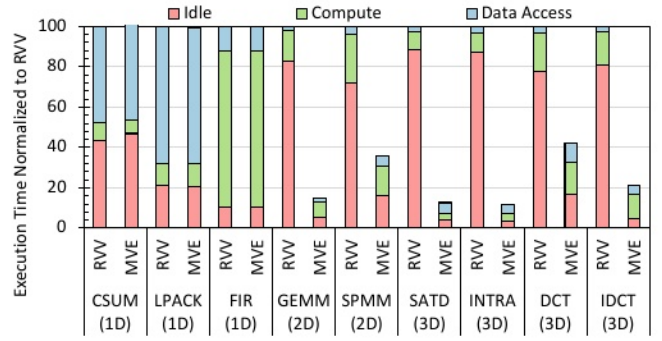


图10。MVE绩效与RISC-V RVV[5]相比，目标是具有8K SIMD通道的相同底层位串行缓存内计算引擎。

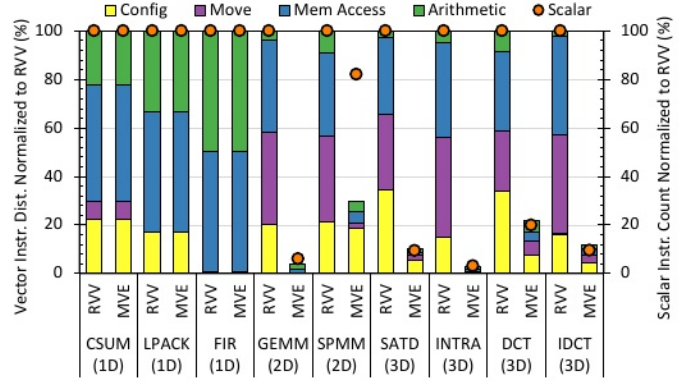


图11。MVE和RISC-V RVV[5]动态向量指令分布(左y轴)和动态标量指令计数(右y轴)。

分1D内存访问，#SIMD车道和移动指令。图11(左y轴)表明，MVE平均将动态向量指令的数量减少了2.3倍。

在XNNPACK库[40]中的各种GEMM内核中，与MVE相比，RVV显著增加了较小输出矩阵(更少的1D并行性)中的动态向量指令。例如，来自MobileNet V1[43]的 128×3136 输出矩阵需要大约 $81923136 \approx 3$ 个掩码、部分内存访问和跨三个1D段(输出行)的移动指令来占用8K SIMD通道。与MVE相比，这导致向量指令计数增加了5.3倍。来自ShuffleNet V2[61]的另一个 122×784 输出矩阵涉及 $8192784 \approx 11$ 个1D段，导致向量指令增加13.0倍。

此外，更多的部分内存访问需要更多的标量地址计算指令。因此，图11(右y轴)显示，与RVV相比，MVE将动态标量指令的数量平均减少了2.0倍。

C. 比较In-SRAM技术

VRAM[9]、EVE[10]和CAPE[19]采用RVV ISA扩展来指导位并行(BP)、位混合(BH)和关联计算(AC)中sram引擎。图13显示，与RVV相比，MVE提高了所有sram内计算方案的绩效。RVV的大量部分内存访问使BS的CB利用率降至23%，BH降至25%，BP降至24%。MVE仅在一个向量指令中有效地编码多维存储器访问，将BS的CB利用率提高到60%，BH

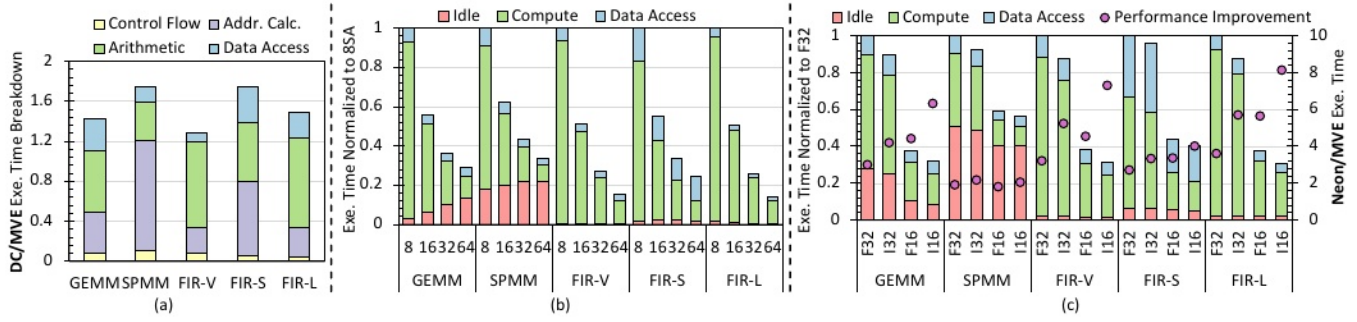


图12. (a)与双缓存[35]的性能比较。(b)使用不同数量的SRAM阵列(SA)的性能可扩展性, 以及(c)对数据精度的性能敏感性。

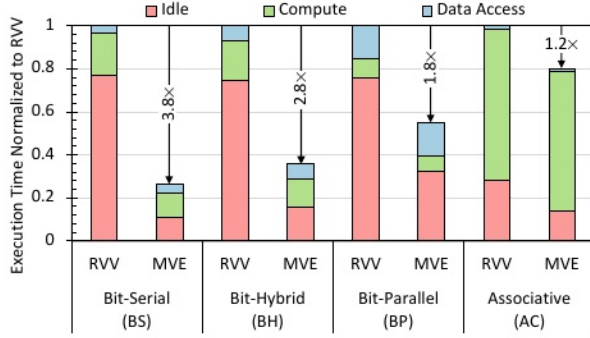


图13. MVE相对RISC-V RVV[5]在VRAM[9]、EVE[10]和CAPE[19]中不同sram内计算方案的绩效改进。

到55%, BP公司到41%。在这些方案中, BP仍显示出相对较低的CB利用率。这是因为BP以较少的SIMD通道为代价享受较少的计算延迟。因此, 缓存内引擎在消耗长向量指令时速度更快, 这就增加了内核以更高速率发出指令的压力。怠速时间的减少使BS、BH和BP的绩效分别提高3.8倍、2.8倍和1.8倍。虽然MVE也将AC空闲时间减少了2.0倍, 但这种in-SRAM计算方案并没有从多维ISA中获得显著好处。这是因为AC算术运算的延迟比BS高4-8倍, 因此70%的AC执行时间花在了算术运算上。

双重性缓存[35]。图12 (a)表明, 使用SIMD模型的MVE实现了比先前基于SIMD的缓存内计算设计双重性缓存[35]平均1.5倍的绩效提升。绩效的提高主要有两个原因: 首先, MVE减少了计算周期, 因为它需要更少的in-SRAM计算操作。在SIMT模型中, 所有操作(控制流、地址计算和算术)都被卸载到in-SRAM计算引擎中, 并由所有SIMT通道单独执行。相比之下, MVE只在标量核上执行一次控制流和基址计算, 而缓存控制器使用MVE内存访问的跨步语义计算每个SIMD通道的内存地址。因此, SIMT模型以更多的in-SRAM操作为代价, 消除了in-SRAM计算引擎的空闲周期。因此, SIMT模型是大型服务器级in-SRAM引擎的理想选择, 其中控

制流发散会带来挑战, 而SIMD模型更适合优先考虑低延迟的小型移动缓存。其次, SIMT模型的数据访问时间比MVE模型长1.6倍。这是因为SIMT模型将所有标量和向量变量保存在稀疏的缓存内物理寄存器中, 导致频繁的寄存器溢出和填充。由于缓存内物理寄存器中有大量的数据元素(8K元素), 这些操作的成本特别高。

D. 可扩展性分析

图12 (b)显示了不同SRAM阵列数下MVE的绩效可扩展性。这些基线的设计是通过保持512 KB二级缓存的完整性, 并使用额外的8到64个专用于MVE的SRAM阵列来增加核心。结果表明, SRAM阵列增加8倍, MVE绩效从3.0 $\times$ (SpMM)提高到6.7 $\times$ (FIR-L)。空闲时间是为大量服务器级缓存扩展MVE绩效的主要瓶颈。

E. 对位精度的敏感性

图12 (c)显示了位精度对MVE执行时间(左y轴)的影响, 以及它相对于Neon(右y轴)的绩效改进, 其中有两个结论: 首先, MVE在较低精度下比Neon表现出更大的绩效改进, 这表明in-SRAM计算从较低的位精度中比基于cmos的处理器更受益。这是因为in-SRAM计算依赖于算术复杂度为 $O(\text{precision}^2)$ 的位串行范式。因此, 位精度对吞吐量有二次效应。相比之下, Neon ASIMD单元以较低的位精度实现线性缩放, 因为它们可以将更多的数据元素装入SIMD寄存器中。其次, 随着精度的降低, MVE的空闲周期不会显著减少。这是因为CBs执行低精度in-SRAM指令的速度要快得多, 超过了内核发出指令的速度。

F. 面积分析

表V评估了MVE与骁龙855移动SoC的面积开销。虽然使用Neon寄存器文件和alu增加标量数据路径会产生16.3%的面积开销, 但MVE将缓存结构转换为长向量引擎, 对标量核心的面积开销可以忽略不计, 为3.6%。这一面积开销与之前的研究结果相似, 例如3.5%[35]和2%[31]。MVE面积为

由FSM控制器控制，将MVE指令解码到 μ ops。该表进一步显示了GPU面积的显著增大。因此，通用MVE设计在可忽略不计的硅面积上提供专用硬件组件的绩效和能源优势，使其成为对成本敏感的移动行业的有吸引力的解决方案。

表5
到标量核心区域的开销(1.07mm² [31])

Module		Source	Area (mm ²)	Overhead (%)
Arm Neon		[21]	0.1741	16.321
MVE	Controller	RTL	0.0043	0.409
	MSHR	CACTI	0.0018	0.168
	TMU	[31]	0.0053	0.498
	XB	[35]	0.0039	0.367
	FSM	[35]	0.0123	1.154
	Peripheral	[35]	0.0063	0.590
	Address Decoder	RTL	0.0042	0.399
	Total	—	0.0382	3.588
Adreno 640 GPU		[41]	11.1908	—

VIII. 相关工作

特定于域的缓存内计算。缓存内计算专门针对不同的领域进行研究，如机器学习[8], [31], [87], 自动机处理[72], [73], [82], 图像处理[16], 和数据加密[89]。然而，特定领域的加速需要额外的努力来定制缓存的数据布局以及核心和缓存元素之间的数据流。像MVE这样的通用执行模型减轻了程序员的负担，为其他领域释放了缓存内计算的能力。

用于缓存内计算的通用ISA设计。有几部作品扩展了一般应用的缓存内计算的范围。我们已经全面讨论了二元缓存[35]、CAPE[19]和EVE[10]。Kooli等人[54], [55]提出了一种用于缓存内计算的通用SIMD模型，将执行流与CPU集成在一起。他们还引入了一种就地shuffle操作来加速密码学、图像处理等。Blade[79]提出了高速缓存内计算在移动处理器的高频和低压下运行的电路级解决方案。这些先前工作的架构贡献与本文是正交的。MVE可以用来提高所有先前的In-SRAM架构的绩效(第VII-B节)。

先前的工作演示了内存数据库的多维访问模式，其中数据可以在逻辑线性地址空间中以列和行方式访问。GS-DRAM[76]提出了DRAM控制器的变化，以独立于多个DRAM芯片获取2次幂跨行访问模式，提高DRAM带宽利用率。MDACache[36]、RC-NVM[56]和MVC[33]修改了计算堆栈和缓存架构，使对非易失性存储器的垂直和水平访问成为可能，从而提高了缓存线利用率。这些方案可以作为MVE的补充，用于提高DRAM内存的带宽。

移动矢量处理。大。VLITTLE[84]在大中重新利用小核。移动处理器的LITTLE架构作为解耦的矢量处理引擎，无需添加矢量寄存器文件和宽执行管道

等面积昂贵的组件。通过重新利用一半的处理器L2缓存，MVE支持更大的SIMD容量，该容量紧密集成到标量核心中。PUMICE[88]提出对缓存中的长向量指令进行乱序调度，以隐藏CAPE[19]的命令分发开销。MVE架构也可以采用这种技术来减少空闲时间。

长向量和矩阵处理。二十年前多媒体扩展的出现激发了增强暴露于SIMD引擎的DLP的设计。MOM[25], [26]提出了二维SIMD指令，同时在第二维支持灵活的向量长度和步长。Breeze[85]和CSI[23]提出了基于cisc的指令，对所有操作、数据访问模式和嵌套循环语义进行编码，以便卸载到流媒体处理器。摩托罗拉56000[52]和德州仪器TMS320C5x [44] dsp采用了复杂的DO循环和跨行地址计算指令，以减少控制流和地址计算开销。最近，Nvidia服务器gpu集成了张量存储器加速器(TMA)单元[66]，能够将1-5D张量从全局存储器批量复制到片上共享存储器。此外，还提出了针对服务器处理器中的机器学习工作负载的ISA扩展。英特尔高级矩阵扩展(AMX)[45]和Arm可扩展矩阵扩展(SME)[13]是2D矩阵ISA扩展，支持高性能的1k位和2k位矩阵操作。AMX有两个主要组件：专门加载固定大小的块(由8个2D寄存器组成，每个寄存器大小为1KB)和块矩阵乘法单元(执行矩阵乘法计算)。

与这些设计相反，MVE是一种长向量ISA扩展，专门针对移动核心的紧耦合缓存内计算引擎，并系统地捕获多维数据并行计算的语义。此外，MVE使用随机内存访问和针对移动工作负载定制的高效维度级掩码执行对不规则计算进行编码。

IX. 结论

这项工作引入了一个长向量多维向量ISA扩展(MVE)，它将缓存内计算抽象为一个紧密集成的长向量处理引擎。MVE通过使用多维跨行和随机内存访问，以及高效的基于维度的掩码执行，实现了大量SIMD通道的高利用率。使用此ISA扩展，MVE将一半的L2移动缓存重新用于缓存内计算，面积开销为3.6%。与商用移动CPU矢量基准相比，长矢量缓存架构提供了2.9倍的绩效提升和8.8倍的能耗降低。与RISC-V RVV相比，MVE将缓存内引擎的利用率从23%提高到60%，在针对相同的底层位串行缓存内计算引擎时，绩效提高了3.8倍。

致谢

这项工作得到了美国国家科学基金会CAREER-1652294和NSF-1908601奖的部分支持，并得到了JST PRESTO资助号JPMJPR22P7的支持。

附录

表vi基准测试实现。

A. 摘要

本文的工件包括用于MVE ISA的缓存内计算模拟器、各种基准测试的实现以及用于基线评估的仪器工具。

B. 工件检查列表(元信息)

- 程序:经过评估的Swan[49]基准的标量、Arm Neon、OpenCL、CUDA、MVE和RVV实现(表VI)。
- 编译:Android NDK r23c 用于 Arm Neon, Adreno OpenCL SDK v1.5 用于 移动 GPU, Clang 3.4+ 用于 模拟器, CMake 3.7+ 用于 DynamoRIO。
- 软件:Git LFS。
- 硬件:三星Galaxy S10e用于封装simd Neon和Adreno GPU基线, Armv8.2-A机器用于生成仿真痕迹, 以及任何用于仿真的系统。
- 输出:由[自动化脚本](#)生成的CSV文件。
- 实验:Arm Neon(移动SIMD)和Adreno(移动GPU)在移动设备上的测量, 位串行(BS)、位混合(BH)、位并行(BP)和关联计算(AC)缓存内计算方案的MVE和RVV模拟。
- 所需磁盘空间:约65gb
- 准备时间:约1小时
- 测量/模拟时间:约24小时
- 公开可用:是的, 在[GitHub](#)和[Zenodo](#)上可用。
- 代码许可:是的, [麻省理工学院](#)许可。
- 工作流自动化:是的, 位于[基准/脚本/](#)。

C. 概述

该工件由两部分组成, 用于再现以下图中所示的结果:(i)图7(a) (MVE与Arm Neon), (ii)图8和9 (MVE与Adreno GPU), 以及(iii)图10、11和13 (MVE与RVV)。第一部分涉及在移动设备上测量, 而第二部分则模拟了BS, BH, BP和AC缓存内计算方案的MVE和RVV内核。这两个部分都需要访问特定的硬件依赖关系, 特别是基于arm的机器。如果您无法访问这些机器, 我们提供预先计算的结果文件来绕过它们。

表VI显示了[这里可用](#)的所有实现。我们为所有库提供标量和Neon实现;然而, 只有Neon实现用于与矢量单位(Arm Neon)进行比较。还为所有库提供了MVE实现。我们选择了10个不同维度的内核, 并实现了以下版本:(a)使用一维指令, 与RVV进行比较;(b)使用OpenCL, 评估Adreno移动GPU上的绩效;(c)使用CUDA, 研究二元缓存(DC)[35]的绩效。请注意, 由于DC的模拟基础设施(GPU Ocelot[29])的复杂性, 该工件不包括图12 (a) (MVE与DC)的复制。

如何访问神器?所有必要的代码、工具和脚本都可以在我们的[GitHub存储库](#)中获得。该存储库还包括大型动态指令和数据流图文件(大约2.5GB), 可以使用它们跳过第2部分(E节)的步骤1和2。如果您希望在下载这些大文件的同时克隆存储库, 请安装Git

Library	#Kernels	Scalar	Neon	MVE	RVV	OpenCL	CUDA
Linpack	1	✓	✓	✓	✓	✓	✓
XNNPACK	2	✓	✓	✓	✓	✓	✓
CMSIS-DSP	3	✓	✓	✓	✓	✓	✓
Kvazaar	4	✓	✓	✓	✓	✓	✓
Arm Opt. Routines	1 4	✓ ✓	✓ ✓	✓ ✓	✓ X	✓ X	✓ X
libjpeg	5	✓	✓	✓	X	X	X
libpng	3	✓	✓	✓	X	X	X
libwebp	7	✓	✓	✓	X	X	X
Skia	4	✓	✓	✓	X	X	X
Webaudio	5	✓	✓	✓	X	X	X
zlib	2	✓	✓	✓	X	X	X
boringssl	3	✓	✓	✓	X	X	X

克隆前的[LFS](#)。否则, 克隆前请设置环境变量GIT_LFS_SKIP_SMUDGE=1。

D. 第1部分-Mobile Measurements

硬件依赖关系。这一步需要配备骁龙855 SoC的三星Galaxy S10e设备。如果没有此设备的访问权限, 可以直接使用“/to/MVE/data/csv_files.tar.gz”路径下预生成的CSV文件跳过此步骤。

软件依赖项。移动测量需要Android NDK r23c([从Android GitHub存储库下载](#))进行交叉编译, Adreno OpenCL SDK v1.5(在[我们的GitHub存储库](#)中提供)和Adreno OpenCL共享库(在[我们的GitHub存储库](#)中提供)用于移动GPU。除此之外, 我们还需要[Android调试桥\(adb\)](#)来启用对手机的shell访问。

安装。安装adb。下载Android NDK r23c、Adreno OpenCL SDK v1.5、Adreno OpenCL共享库并解压。运行下面的bash脚本来设置交叉编译基础架构, 并编译Arm Neon和Adreno GPU基线。

```
export ANDROID_NDK_ROOT=path/to/android-ndk-r23c
cp -r path/to/MVE/tools/opencv-sdk-1.5/inc/CL
$ANDROID_NDK_ROOT/toolchains/llvm/prebuilt/linux
-x86_64/sysroot/usr/include/
cp path/to/MVE/tools/libopencv/libOpenCL.so
$ANDROID_NDK_ROOT/toolchains/llvm/prebuilt/linux
-x86_64/lib/
cd path/to/MVE/benchmarks
make phone_neon -j
make phone_adreno -j
```

实验工作流。将三星S10e设备连接到您的计算机。启用[开发者选项](#)和[USB调试](#)。运行以下提供的脚本, 自动生成neng.csv和adreno.csv结果。

```
cd path/to/MVE/benchmarks
python scripts/profiler.py --core prime \
--measurement performance --platform neon \
--output neon.csv --verbose
python scripts/profiler.py --core prime \
--measurement performance --platform adreno \
--output adreno.csv --verbose
```

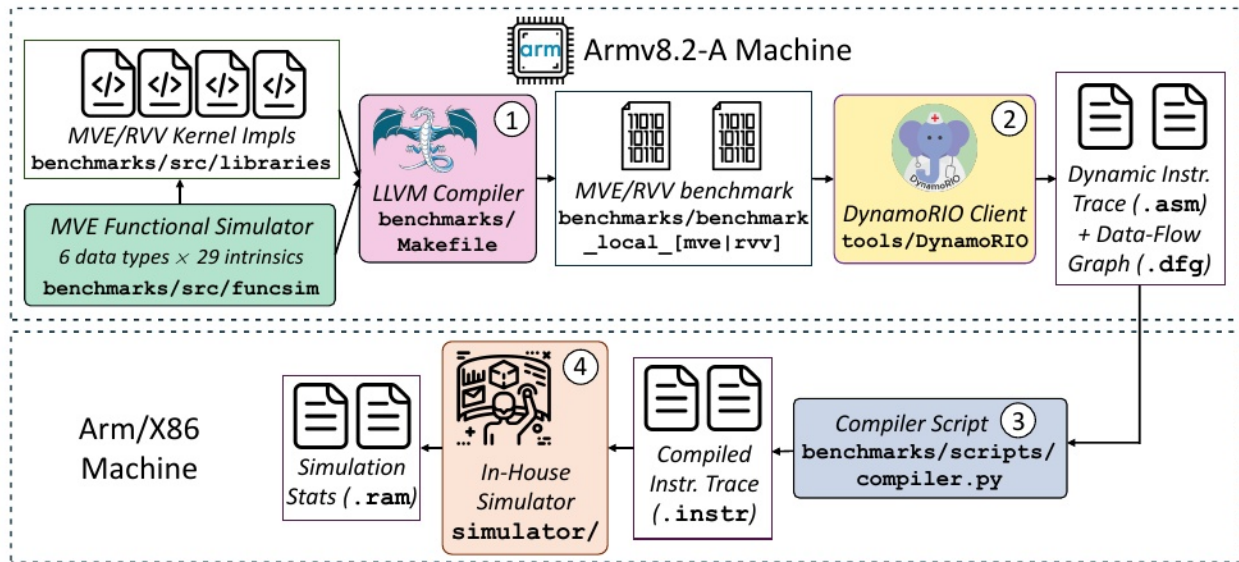


图14.仿真分为四个步骤。步骤1和2必须在Armv8.2-A机器上执行。步骤3和步骤4可以在任何机器上执行。

E. Part 2 -In-Cache Simulation

概述。图14说明了仿真过程的四个步骤。步骤1、2和3生成指令跟踪，而最后的步骤4运行模拟。

硬件依赖关系。步骤1和2需要一台配备Armv8.2-A ISA的机器。剩下的步骤3和步骤4可以在任何机器上执行。如果你没有访问这台机器的权限，你可以跳过前两个步骤。

软件依赖关系。我们实现了一个DynamoRIO客户端[42]来检测基准测试并转储动态指令跟踪。我们使用内部模拟器进行缓存内计算模拟。

安装。使用以下脚本在具有Armv8.2-A ISA的机器上安装DynamoRIO客户端。如果你没有这样一台机器的访问权限，你可以跳过这一步。

```
cd path/to/MVE/tools/DynamoRIO/samples
mkdir build
cd build
cmake ..
make -j
```

要安装我们的模拟器，请运行以下脚本。请注意，即使您打算跳过第2部分的前两个步骤，构建模拟器也是强制性的。

```
cd path/to/MVE/simulator
bash make_all.sh
```

实验工作流程。使用下面的bash脚本，模拟分4步进行。步骤1构建MVE和RVV基线。第2步运行MVE和RVV基准测试，并使用DynamoRIO客户端为每个内核生成动态指令跟踪(.asm)和数据流图(.dfg)文件。步骤3将这些文件编译成模拟器的跟踪文件。第4步运行模拟器并生成仿真结果(.ram)。要配置并发模拟的数量，可以使用-j [NUM_THREADS]参数。

最后，提供了脚本来解析仿真结果并生成任意ISA (MVE和RVV)和缓存内计算方案(BS, BH, BP, AC)组合的CSV文件。

```
cd path/to/MVE/benchmarks
# Step 1: building the MVE and RVV kernels
make local_mve -j
make local_rvv -j
source ./set_env.sh
# Step 2: running DynamoRIO's instrumentation tool
python scripts/simulator.py --action benchmark \
--directory ./data --verbose
# Step 3: compiling asm and dfg files into traces
python scripts/simulator.py --action compile \
--directory ./data --verbose
# Step 4: running the simulations
python scripts/simulator.py --action simulate \
--directory ./data -j [NUM_THREADS] --verbose
# Parsing the results
python scripts/simulator.py --action parse \
--directory ./data --verbose
```

跳过步骤1和2。使用生成的(.asm)和(.dfg)文件跳过前两步。首先，确保下载了这些文件(参见C节-如何访问artifact?)。接下来，执行以下操作提取这些文件并继续步骤3。

```
cd path/to/MVE/benchmarks
tar -xvzf ../data/data_bs.tar.gz
tar -xvzf ../data/data_bh.tar.gz
tar -xvzf ../data/data_bp.tar.gz
tar -xvzf ../data/data_ac.tar.gz
# Resume Step 3
```

F. 评估和预期结果

预生成的CSV文件位于/to/MVE/data/csv_files.tar.gz路径下。使用这些文件填写结果Excel文件的前10页，该文件位于路径/to/MVE/data/Results.xlsx。完成这些表格将生成图7(a)、8、9、10、11和13。

参考文献

- [1] Android - secure and reliable mobile operating system. [Online]. Available: <https://www.android.com/>
- [2] Chromium. [Online]. Available: <https://www.chromium.org/Home/>
- [3] Kirin 990 5g dieshot. [Online]. Available: <https://xian333c.top/archives/526>
- [4] Pdfium. [Online]. Available: <https://pdfium.googlesource.com/pdfium/+master/README.md>
- [5] riscv-v-spec. [Online]. Available: <https://github.com/riscv/riscv-v-spec>
- [6] WebRTC. [Online]. Available: <https://webrtc.org/>
- [7] M. Adler and J.-I. Gailly. Zlib data compression library. [Online]. Available: <https://github.com/madler/zlib>
- [8] S. Aga, S. Jeloka, A. Subramaniyan, S. Narayanasamy, D. Blaauw, and R. Das, "Compute caches," in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 481–492.
- [9] K. Al-Hawaj, T. Ta, N. Cebry, S. Agwa, O. Afuye, E. Hall, C. Golden, A. B. Apsel, and C. Batten, "Towards a reconfigurable bit-serial/bit-parallel vector accelerator using in-situ processing-in-sram," in *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2020, pp. 1–5.
- [10] K. Al-Hawaj, T. Ta, N. Cebry, S. Agwa, O. Afuye, E. Hall, C. Golden, A. B. Apsel, and C. Batten, "Eve: Ephemeral vector engines," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 691–704.
- [11] Arm. Arm cortex-a76 core technical reference manual r3p0. [Online]. Available: <https://developer.arm.com/documentation/100798/0300/>
- [12] Arm. Arm optimized routines. [Online]. Available: <https://github.com/ARM-software/optimized-routines>
- [13] Arm. Introducing the scalable matrix extension for the armv9-a architecture. [Online]. Available: <https://community.arm.com/arm-community-blogs/b/architectures-and-processors-blog/posts/scalable-matrix-extension-armv9-a-architecture>
- [14] Arm-Software. Cmsis-dsp, an optimized compute library for embedded systems. [Online]. Available: <https://github.com/ARM-software/CMSIS-DSP>
- [15] I. A. Assir, M. E. Iskandarani, H. R. A. Sandid, and M. A. R. Saghir, "Arrow: A risc-v vector accelerator for machine learning inference," 2021. [Online]. Available: <https://arxiv.org/abs/2107.07169>
- [16] S. K. Bose, V. Mohan, and A. Basu, "A 75kb sram in 65nm cmos for in-memory computing based neuromorphic image denoising," in *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2020, pp. 1–5.
- [17] A. Cabrera, S. Hitefield, J. Kim, S. Lee, N. R. Miniskar, and J. S. Vetter, "Toward performance portable programming for heterogeneous systems on a chip: A case study with qualcomm snapdragon soc," in *2021 IEEE High Performance Extreme Computing Conference (HPEC)*, 2021, pp. 1–7.
- [18] H. Caminal, Y. Chronis, T. Wu, J. M. Patel, and J. F. Martínez, "Accelerating database analytic query workloads using an associative processor," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 623–637. [Online]. Available: <https://doi.org/10.1145/3470496.3527435>
- [19] H. Caminal, K. Yang, S. Srinivasa, A. K. Ramanathan, K. Al-Hawaj, T. Wu, V. Narayanan, C. Batten, and J. F. Martínez, "Cape: A content-addressable processing engine," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 557–569.
- [20] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, "Ara: A 1-ghz+ scalable and energy-efficient risc-v vector processor with multiprecision floating-point support in 22-nm fd-soi," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 2, pp. 530–543, 2020.
- [21] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, "Ara: A 1-ghz+ scalable and energy-efficient risc-v vector processor with multiprecision floating-point support in 22-nm fd-soi," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 2, pp. 530–543, 2020.
- [22] C. Chen, X. Xiang, C. Liu, Y. Shang, R. Guo, D. Liu, Y. Lu, Z. Hao, J. Luo, Z. Chen, C. Li, Y. Pu, J. Meng, X. Yan, Y. Xie, and X. Qi, "Xuantie-910: A commercial multi-core 12-stage pipeline out-of-order 64-bit high performance risc-v processor with vector extension: Indus-trial product," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 52–64.
- [23] D. Cheresiz, B. Juurlink, S. Vassiliadis, and H. Wijshoff, "The csi multimedia architecture," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 13, no. 1, pp. 1–13, 2005.
- [24] cMathLibraries. A software library containing sparse functions written in opencl. [Online]. Available: <https://github.com/cMathLibraries/cSPARSE/>
- [25] J. Corbal, R. Espasa, and M. Valero, "Mom: a matrix simd instruction set architecture for multimedia applications," in *SC '99: Proceedings of the 1999 ACM/IEEE Conference on Supercomputing*, 1999, pp. 15–15.
- [26] J. Corbal, R. Espasa, and M. Valero, "Three-dimensional memory vectorization for high bandwidth media memory systems," in *35th Annual IEEE/ACM International Symposium on Microarchitecture*, 2002. (MICRO-35). *Proceedings.*, 2002, pp. 149–160.
- [27] M. Demler, "Andes plots risc-v vector heading," *Microprocessor Report*, 2020.
- [28] A. Developers. Profile battery usage with batterystats and battery historian. [Online]. Available: <https://developer.android.com/topic/performance/power/setup-battery-historian>
- [29] G. Diamos, A. Kerr, S. Yalamanchili, and N. Clark, "Ocelot: A dynamic optimization framework for bulk-synchronous applications in heterogeneous systems," in *2010 19th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2010, pp. 353–364.
- [30] J. J. Dongarra, P. Luszczek, and A. Petitet, "The linpack benchmark: past, present and future," *Concurrency and Computation: practice and experience*, vol. 15, no. 9, pp. 803–820, 2003.
- [31] C. Eckert, X. Wang, J. Wang, A. Subramaniyan, D. Sylvester, D. Blaauw, R. Das, and R. Iyer, "Neural cache: Bit-serial in-cache acceleration of deep neural networks," *IEEE Micro*, vol. 39, no. 3, p. 11–19, may 2019. [Online]. Available: <https://doi.org/10.1109/MM.2019.2908101>
- [32] C. N. Fischer and R. J. LeBlanc Jr, *Crafting a Compiler with C*. Benjamin-Cummings Publishing Co., Inc., 1991.
- [33] D. Fujiki, "Mvc: Enabling fully coherent multi-data-views through the memory hierarchy with processing in memory," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 800–814.
- [34] D. Fujiki, A. Khadem, S. Mahlke, and R. Das, "Multi-layer in-memory processing," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 920–936.
- [35] D. Fujiki, S. Mahlke, and R. Das, "Duality cache for data parallel acceleration," in *Proceedings of the 46th International Symposium on Computer Architecture*, ser. ISCA '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 397–410. [Online]. Available: <https://doi.org/10.1145/3307650.3322257>
- [36] S. George, M. J. Liao, H. Jiang, J. B. Kotra, M. T. Kandemir, J. Sampson, and V. Narayanan, "Mdacache: Caching for multi-dimensional-access memories," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018, pp. 841–854.
- [37] Google. Blink (rendering engine). [Online]. Available: <https://www.chromium.org/blink/>
- [38] Google. Boringssl library. [Online]. Available: <https://github.com/google/boringssl>
- [39] Google. Skia - a complete 2d graphic library for drawing text, geometries, and images. [Online]. Available: <https://github.com/google/skia>
- [40] Google. Xnnpack, a highly optimized library of floating-point neural network inference operators. [Online]. Available: <https://github.com/google/XNNPACK>
- [41] gsmarena. Comparing the sizes of flagship chipsets: Qualcomm, samsung, huawei and apple. [Online]. Available: https://www.gsmarena.com/comparing_the_sizes_of_flagship_chipsets_qualcomm_samsung_huawei_adn_apple-news-30240.php
- [42] B. Hawkins, B. Demsky, D. Bruening, and Q. Zhao, "Optimizing binary translation of dynamically generated code," in *Proceedings of the 13th Annual IEEE/ACM International Symposium on Code Generation and Optimization*, ser. CGO '15. USA: IEEE Computer Society, 2015, p. 68–78.
- [43] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," 2017. [Online]. Available: <https://arxiv.org/abs/1704.04861>
- [44] T. Instruments. Tms320c5x user's guide. [Online]. Available: <https://www.ti.com/lit/ug/spru056d/spru056d.pdf>
- [45] Intel. Accelerate artificial intelligence (ai) workloads with intel advanced matrix extensions (intel amx). [Online].

Available: <https://www.intel.com/content/www/us/en/content-details/785250/accelerate-artificial-intelligence-ai-workloads-with-intel-advanced-matrix-extensions-intel-amx.html>

- [46] S. Jeloka, N. B. Akesh, D. Sylvester, and D. Blaauw, "A 28 nm configurable memory, (tcam/bcam/sram) using push-rule 6t bit cell enabling logic-in-memory," *IEEE Journal of Solid-State Circuits*, vol. 51, no. 4, pp. 1009–1021, 2016.
- [47] S. Jeloka, N. B. Akesh, D. Sylvester, and D. Blaauw, "A 28 nm configurable memory, (tcam/bcam/sram) using push-rule 6t bit cell enabling logic-in-memory," *IEEE Journal of Solid-State Circuits*, vol. 51, no. 4, pp. 1009–1021, 2016.
- [48] M. Kang, E. P. Kim, M.-s. Keel, and N. R. Shanbhag, "Energy-efficient and high throughput sparse distributed memory architecture," in *2015 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2015, pp. 2505–2508.
- [49] A. Khadem, D. Fujiki, N. Talati, S. Mahlke, and R. Das, "Vector-processing for mobile devices: Benchmark and analysis," in *2023 IEEE International Symposium on Workload Characterization (IISWC)*, 2023, pp. 15–27.
- [50] S. Kim, "Reducing area overhead for error-protecting large l2/l3 caches," *IEEE Transactions on Computers*, vol. 58, no. 3, pp. 300–310, 2009.
- [51] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer Architecture Letters*, vol. 15, no. 1, pp. 45–49, 2016.
- [52] K. L. Kloker, "The motorola dsp56000 digital signal processor," *IEEE Micro*, vol. 6, no. 6, pp. 29–48, 1986.
- [53] K. Komatsu, S. Momose, Y. Isobe, O. Watanabe, A. Musa, M. Yokokawa, T. Aoyama, M. Sato, and H. Kobayashi, "Performance evaluation of a vector supercomputer sx-aurora tsubasa," in *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2018, pp. 685–696.
- [54] M. Kooli, H.-P. Charles, C. Touzet, B. Giraud, and J.-P. Noel, "Smart instruction codes for in-memory computing architectures compatible with standard sram interfaces," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2018, pp. 1634–1639.
- [55] M. Kooli, A. Heraud, H.-P. Charles, B. Giraud, R. Gauchi, M. Ezzadeen, K. Mambu, V. Egloff, and J.-P. Noel, "Towards a truly integrated vector processing unit for memory-bound applications based on a cost-competitive computational sram design solution," *J. Emerg. Technol. Comput. Syst.*, vol. 18, no. 2, apr 2022. [Online]. Available: <https://doi.org/10.1145/3485823>
- [56] S. Li, N. Xiao, P. Wang, G. Sun, X. Wang, Y. Chen, H. H. Li, J. Cong, and T. Zhang, "Rc-nvm: Dual-addressing non-volatile memory architecture supporting both row and column memory accesses," *IEEE Transactions on Computers*, vol. 68, no. 2, pp. 239–254, 2019.
- [57] libjpeg_turbo. libjpeg-turbo. [Online]. Available: <https://libjpeg-turbo.org/>
- [58] libpng. official png reference library. [Online]. Available: <http://www.libpng.org/pub/png/libpng.html>
- [59] libwebp. Webp codec. [Online]. Available: <https://github.com/webmproject/libwebp>
- [60] B. C. Lopes and R. Auler, *Getting started with LLVM core libraries*. Packt Publishing Ltd, 2014.
- [61] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "Shufflenet v2: Practical guidelines for efficient cnn architecture design," 2018. [Online]. Available: <https://arxiv.org/abs/1807.11164>
- [62] F. Minervini and O. Palomar, "Vitruvius: And area-efficient risc-v decoupled vector accelerator for high performance computing," *RISC-V Summit*, 2021.
- [63] F. Minervini, O. Palomar, O. Unsal, E. Reggiani, J. Quiroga, J. Marimon, C. Rojas, R. Figueras, A. Ruiz, A. Gonzalez, J. Mendoza, I. Vargas, C. Hernandez, J. Cabre, L. Khoirunisya, M. Bouhali, J. Pavon, F. Moll, M. Olivieri, M. Kovac, M. Kovac, L. Dragic, M. Valero, and A. Cristal, "Vitruvius+: An area-efficient risc-v decoupled vector coprocessor for high performance computing applications," *ACM Trans. Archit. Code Optim.*, vol. 20, no. 2, mar 2023. [Online]. Available: <https://doi.org/10.1145/3575861>
- [64] N. Muralimanohar, R. Balasubramanian, and N. P. Jouppi, "Cacti 6.0: A tool to model large caches," *HP laboratories*, vol. 27, p. 28, 2009.
- [65] C. Nugteren, "Clblast: A tuned opencl blas library," in *Proceedings of the International Workshop on OpenCL*, ser. IWOCL '18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: <https://doi.org/10.1145/3204919.3204924>
- [66] Nvidia. Nvidia hopper architecture in-depth. [Online]. Available: <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>
- [67] J. S. Olesen. Greedy register allocation in llvm 3.0. [Online]. Available: <http://blog.llvm.org/2011/09/greedy-register-allocation-in-llvm-30.html>
- [68] M. Platzner and P. Puschner, "Vicuna: A Timing-Predictable RISC-V Vector Coprocessor for Scalable Parallel Computation," in *33rd Euromicro Conference on Real-Time Systems (ECRTS 2021)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), B. B. Brandenburg, Ed., vol. 196. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, pp. 1:1–1:18. [Online]. Available: <https://drops.dagstuhl.de/opus/volltexte/2021/13932>
- [69] qualcomm. Hexagon npu sdk. [Online]. Available: <https://www.qualcomm.com/developer/software/hexagon-npu-sdk>
- [70] Qualcomm. Introducing trepn profiler 6.0. [Online]. Available: <https://developer.qualcomm.com/blog/introducing-trepn-profiler-60>
- [71] Qualcomm. Snapdragon mobile platform opencl general programming and optimization guide. [Online]. Available: <https://developer.qualcomm.com/download/adrenosdk/adreno-opencl-programming-guide.pdf>
- [72] E. Sadredini, R. Rahimi, M. Lenjani, M. Stan, and K. Skadron, "Impala: Algorithm/architecture co-design for in-memory multi-stride pattern matching," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 86–98.
- [73] E. Sadredini, R. Rahimi, V. Verma, M. Stan, and K. Skadron, "Eap: A scalable and efficient in-memory accelerator for automata processing," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, p. 87–99. [Online]. Available: <https://doi.org/10.1145/3352460.3358324>
- [74] M. Sato, Y. Ishikawa, H. Tomita, Y. Kodama, T. Odajima, M. Tsuji, H. Yashiro, M. Aoki, N. Shida, I. Miyoshi, K. Hirai, A. Furuya, A. Asato, K. Morita, and T. Shimizu, "Co-design for a64fx manycore processor and "fugaku", in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020, pp. 1–15.
- [75] C. Schmidt, J. Wright, Z. Wang, E. Chang, A. Ou, W. Bae, S. Huang, V. Milovanović, C. A. Flynn, B. Richards et al., "An eight-core 1.44-ghz risc-v vector processor in 16-nm finfet," *IEEE Journal of Solid-State Circuits*, vol. 57, no. 1, pp. 140–152, 2021.
- [76] V. Seshadri, T. Mullins, A. Boroumand, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. C. Mowry, "Gather-scatter dram: In-dram address translation to improve the spatial locality of non-unit strided accesses," in *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2015, pp. 267–280.
- [77] SiFive. Intelligence x280. [Online]. Available: <https://www.sifive.com/document-file/x280-datasheet>
- [78] SiFive. Performance p270. [Online]. Available: <https://www.sifive.com/document-file/x280-datasheet>
- [79] W. A. Simon, Y. M. Qureshi, M. Rios, A. Levisse, M. Zapater, and D. Aienza, "Blade: An in-cache computing architecture for edge devices," *IEEE Transactions on Computers*, vol. 69, no. 9, pp. 1349–1363, 2020.
- [80] N. Stephens, S. Biles, M. Boettcher, J. Eapen, M. Eyole, G. Gabrielli, M. Horsnell, G. Magklis, A. Martinez, N. Premillieu, A. Reid, A. Rico, and P. Walker, "The arm scalable vector extension," *IEEE Micro*, vol. 37, no. 2, pp. 26–39, 2017.
- [81] A. Stillmaker and B. Baas, "Scaling equations for the accurate prediction of cmos device performance from 180nm to 7nm," *Integration*, vol. 58, pp. 74–81, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167926017300755>
- [82] A. Subramaniyan, J. Wang, E. R. M. Balasubramanian, D. Blaauw, D. Sylvester, and R. Das, "Cache automaton," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-50 '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 259–272. [Online]. Available: <https://doi.org/10.1145/3123939.3123986>
- [83] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the high efficiency video coding (hevc) standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 22, no. 12, pp. 1649–1668, 2012.
- [84] T. Ta, K. Al-Hawaj, N. Cebry, Y. Ou, E. Hall, C. Golden, and C. Batten, "big_vllittle: On-demand data-parallel acceleration for mobile systems on chip," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 181–198.

- [85] D. Talla, L. John, and D. Burger, "Bottlenecks in multimedia processing with simd style extensions and architectural enhancements," *IEEE Transactions on Computers*, vol. 52, no. 8, pp. 1015–1031, 2003.
- [86] M. Viitanen, A. Koivula, A. Lemmetti, A. Ylä-Outinen, J. Vanne, and T. D. Hämäläinen, "Kvazaar: Open-source hevc/h.265 encoder," in *Proceedings of the 24th ACM International Conference on Multimedia*, ser. MM '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 1179–1182. [Online]. Available: <https://doi.org/10.1145/2964284.2973796>
- [87] X. Wang, J. Yu, C. Augustine, R. Iyer, and R. Das, "Bit prudent in-cache acceleration of deep convolutional neural networks," in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2019, pp. 81–93.
- [88] S. Wong, C. C. Tamarit, and J. F. Martínez, "Pumice: Processing-using-memory integration with a scalar pipeline for symbiotic execution," in *Design Automation Conference (DAC)*, july 2023.
- [89] J. Zhang, H. Naghibijouybari, and E. Sadredini, "Sealer: In-sram aes for high-performance and low-overhead memory encryption," in *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*, ser. ISLPED '22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: <https://doi.org/10.1145/3531437.3539699>